



Proyecto Jago — Guía Completa Paso a Paso

Asistente virtual con avatar (la mascota jaguar) para el Open House del Saint Margaret School.

Este documento explica **TODO** lo que se necesitó para crear a Jago: cada **herramienta, app, cuenta y tecnología** (casi todo **gratis**), qué hace cada una, y **cómo se construyó paso a paso**, desde cero hasta publicarlo en internet con base de datos en la nube e inteligencia artificial.

🎯 **Qué hace Jago:** es un jaguar en una página web que **saluda, escucha por micrófono, responde hablando, mueve la boca y la cola, pregunta el nombre y el grado del hijo del visitante, recomienda los proyectos de ese grado, responde con IA (Gemini) y guarda a cada visitante en una base de datos en la nube**. Está **publicado en internet** y se comparte con un **código QR**.

🌐 **En vivo:** <https://hays05.github.io/feria-jaguar/>



Índice

1. Resumen: de qué está hecho Jago
2. Cuentas y servicios que se necesitaron
3. Tabla COMPLETA de herramientas y tecnologías
4. Mapa de archivos: qué hace cada uno
5. Cómo arrancar el proyecto en tu compu
6. Cómo se construyó, paso a paso (todas las fases)
7. Cómo se conectaron los servicios (Gemini, GitHub Pages, Supabase)
8. Seguridad: cómo se protegió
9. Cómo reproducirlo desde cero (resumen)
10. Glosario para principiantes
11. Errores comunes y soluciones

1) Resumen: de qué está hecho Jago

Jago es una **página web** hecha con las 3 tecnologías base de la web — **HTML, CSS y JavaScript** — **sin frameworks** (sin React, sin librerías pesadas). A eso se le sumaron servicios gratuitos para darle superpoderes:

- **Voz** (oír y hablar) → la trae el propio navegador (**Web Speech API**).
- **Inteligencia artificial** → **Google Gemini**.
- **Base de datos en la nube** → **Supabase**.
- **Publicación en internet** → **GitHub + GitHub Pages**.
- **Animación de boca y cola** → **SVG + CSS** + una imagen recortada con **Python**.

2) Cuentas y servicios que se necesitaron

Para construir y publicar a Jago se usaron estas **cuentas** (todas con plan gratuito):

Servicio	¿Para qué se usó?	Cuenta	Enlace
GitHub	Guardar el código en la nube y publicar la página (GitHub Pages)	HAYS05	https://github.com
Google AI Studio	Generar la API Key de Gemini (la IA que responde)	tu cuenta Google	https://aistudio.google.com/apikey
Supabase	Base de datos en la nube (guardar a todos los visitantes)	proyecto feria-jaguar	https://supabase.com

Y estos **programas instalados en la computadora** (gratis):

Programa	¿Para qué?	Enlace
Google Chrome / Microsoft Edge	Ver la página y usar voz/micrófono (la voz funciona mejor en estos)	https://google.com/chrome
Visual Studio Code	Editor donde se escribe el código	https://code.visualstudio.com
Python	1) Servidor local para probar; 2) recortar la cola del jaguar (con Pillow)	https://python.org
Git	Control de versiones (guardar cada cambio y subir a GitHub)	https://git-scm.com

💡 La voz, el guardado local y la animación **no necesitan cuenta ni internet**. Solo Gemini y Supabase necesitan internet y cuenta.

3) Tabla COMPLETA de herramientas y tecnologías

Tecnología / Herramienta	¿Qué hace en Jago?	¿Gratis?	Dónde se usa
HTML	Estructura de la página (títulos, botones, chat, cajas)	Sí	<code>index.html</code> , <code>cartel.html</code>
CSS	Diseño, colores, layout de 3 columnas y animaciones (boca, cola, respiración)	Sí	<code>css/estilos.css</code>
JavaScript (vanilla)	Toda la lógica: hablar, escuchar, chat, guardar, IA	Sí	carpeta <code>js/</code>
Web Speech API — SpeechSynthesis	Texto a voz : que Jago HABLE	Sí (navegador)	<code>js/avatar.js</code>
Web Speech API — SpeechRecognition	Voz a texto : que Jago ESCUCHE el micrófono	Sí (navegador)	<code>js/voz.js</code>
SVG	Dibujo de la boca (labio, cavidad, dientes, colmillos, lengua) y el fondo tecnológico	Sí	dentro de <code>index.html</code> y <code>assets/fondo-tech.svg</code>
localStorage	Base de datos local del navegador (respaldo de visitantes)	Sí	<code>js/base-datos.js</code>
Google Gemini API (<code>gemini-2.5-pro</code> / <code>flash</code>)	IA que responde preguntas libres como Jago	Plan gratuito + de pago	<code>js/ia-gemini.js</code>
Supabase (Postgres + API REST/PostgREST + RLS)	Base de datos en la nube : junta a todos los visitantes en un solo lugar	Sí (nivel gratuito)	<code>js/nube.js</code> , <code>datos/supabase-feria.sql</code>
GitHub	Repositorio (guardar el código y su historial)	Sí	repo <code>HAYS05/feria-jaguar</code>
GitHub Pages	Hosting : publica la página en internet gratis	Sí	<code>https://hays05.github.io/feria-jaguar/</code>
Git	Control de versiones (cada cambio queda guardado)	Sí	todo el proyecto
Python + Pillow (PIL)	Recortar la cola del jaguar de la foto y crear la capa transparente que se anima	Sí	<code>_tail_tool.py</code>
API de códigos QR (goqr.me)	Generar el código QR que abre la página	Sí	imagen <code>assets/qr-feria.png</code>
Python <code>http.server</code>	Servidor local para probar antes de publicar	Sí	<code>INICIAR.bat</code>

Herramientas que se compararon pero NO se usaron (y por qué)

Herramienta	Para qué sirve	Por qué no se usó
ElevenLabs	Voz súper realista	De pago para uso amplio; la voz del navegador es gratis
Vercel / Netlify	Hosting de apps con servidor	La página es estática → GitHub Pages basta y es más simple
React / Next.js	Framework de interfaz	Para principiantes, HTML+CSS+JS puro es más fácil de entender
Live2D / avatares 3D	Avatares profesionales animados	Mucho trabajo; cambiaría la mascota del colegio

4) Mapa de archivos: qué hace cada uno

FeriaJaguar/	
├ index.html	← LA PÁGINA (lo que se ve): cabecera, 3 columnas, chat, modal de stands
├ cartel.html	← Cartel imprimible con el QR para pegar en el evento
├ config.js	← Configuración: modelo de Gemini + datos de Supabase
├ config.ejemplo.js	← Plantilla de config (sin datos reales)
├ INICIAR.bat	← Doble clic para arrancar el servidor local (Windows)
├ LEEME.txt	← Guía rápida
├ README.md	← Presentación del repositorio
├ .gitignore	← Qué NO se sube a GitHub (backups, etc.)
├ _tail_tool.py	← Script de Python que recortó la cola del jaguar
├ css/	
└ estilos.css	← Diseño, layout, colores y animaciones (boca, cola, respiración)
├ js/	← El "cerebro" dividido por tareas
└ avatar.js	← Habla, mueve la boca, maneja el chat y el SALUDO guiado
└ cerebro.js	← Entiende preguntas y elige respuesta (modo offline)
└ voz.js	← Escucha por micrófono (voz a texto)
└ feria.js	← Galería de stands + resumir un stand
└ ia-gemini.js	← Conexión con la IA (Gemini) + respaldo automático
└ nube.js	← Guarda visitantes y mensajes en Supabase (la nube)
└ base-datos.js	← Guarda visitantes en el navegador (localStorage, respaldo)
├ datos/	
└ conocimiento.json	← LO QUE SABE Jago (evento, stands, proyectos por grado, FAQs)
└ supabase-feria.sql	← Script para crear las tablas en la nube
└ INFO-QUE-NECESITO.md	← Plantilla para recolectar la info real del evento
├ assets/	
└ mascota.jpg	← Foto del jaguar (cuerpo)
└ jaguar-cola.png	← La COLA recortada (capa transparente que se anima)
└ logo-sms.png	← Logo del colegio (círculo blanco)
└ fondo-tech.svg	← Fondo tipo circuito (decoración tecnológica)
└ qr-feria.png	← Código QR que abre la página
└ documentacion/	
├ GUIA-EQUIPOS.md	← Qué construye cada equipo
├ GUIA-PUBLICAR.md	← Cómo se publicó en GitHub Pages
└ GUIA-COMPLETA-PASO-A-PASO.md	← (este documento)

💡 **Idea clave:** el código está **separado por tareas**. Así cada equipo trabaja en SU archivo sin estorbar a los demás. Los archivos JS se cargan en orden al final de `index.html`.

5) Cómo arrancar el proyecto en tu compu

Forma A — La más fácil (Windows)

1. Abre la carpeta `FeriaJaguar`.
2. Doble clic en `INICIAR.bat` → se abre una ventana negra (no la cierres) y el navegador.
3. Si no abre solo, entra a `http://localhost:8000`.

Forma B — Desde VS Code

1. Abre la carpeta en VS Code.
2. **Terminal** → **New Terminal** → escribe `python -m http.server 8000` → Enter.
3. Abre `http://localhost:8000`.

⚠️ ¿Por qué un servidor y no doble clic al `index.html`? Porque el navegador, por seguridad, bloquea cargar el archivo de conocimiento y el micrófono cuando se abre como archivo suelto. El servidor local (`localhost`) lo resuelve.

6) Cómo se construyó, paso a paso (todas las fases)

Cada fase se hizo encima de la anterior. La idea pedagógica: empezar simple y agregar superpoderes uno por uno.

● Fase 1 — El jaguar aparece y HABLA

Una página con la imagen del jaguar y un botón **"Saludar"**. Al presionarlo, habla en voz alta con la voz del navegador.

```
const voz = new SpeechSynthesisUtterance("¡Hola! Soy Jago");
voz.lang = "es-ES";
window.speechSynthesis.speak(voz);
```

● Fase 2 — Responde preguntas (chat + cerebro local)

Un chat: escribes y Jago responde por texto y voz. Su "cerebro" es `datos/conocimiento.json` (saludos, evento, stands, FAQs). `js/cerebro.js` compara tu pregunta con **palabras clave** y elige la mejor respuesta. Antes de comparar, **normaliza** (quita mayúsculas y acentos) para que "¿A qué HORA?" = "a que hora".

● Fase 3 — Escucha por micrófono

Botón 🎤. Usa **SpeechRecognition** del navegador para convertir tu voz en texto y responder.

```
const oido = new webkitSpeechRecognition();
oido.lang = "es-ES";
oido.onresult = (e) => { caja.value = e.results[0][0].transcript; enviarPregunta(); };
oido.start();
```

El micrófono solo funciona en `localhost` o `https`, y pide permiso la primera vez.

● Fase 4 — Mueve la boca + respiración

La boca está dibujada en **SVG** (labio, cavidad con degradado, dientes, colmillos, lengua). Se **sincroniza con el sonido**: se abre cuando arranca la voz (`onstart`) y se cierra al terminar (`onend`), moviéndose **suavemente** hacia un objetivo que cambia con cada palabra. La imagen y la boca van dentro de `.cara` para moverse juntas, con una animación de **respiración** y leve balanceo. La posición de la boca se ajusta con variables CSS: `--boca-x` , `--boca-y` , `--boca-ancho` , `--boca-alto` .

● Fase 5 — Base de datos local de visitantes

Guarda a cada persona con **localStorage** (la "cajita" del navegador): nombre, hora y todas las preguntas/respuestas. El **Panel del profesor** permite descargar todo en **Excel (CSV)** o **JSON**. (Más tarde se sumó la nube.)

● Fase 6 — Galería de stands + resumir

Botón **"Ver stands"** que abre una ventana (modal) con una **tarjeta por stand**. Al tocar una, Jago la **resume en voz alta**. La función `resumir()` acorta el texto largo a 1-2 frases.

● Fase 7 — Conectar Gemini (IA real)

`js/ia-gemini.js` envía la pregunta a **Google Gemini** junto con la info del evento (para que **no invente datos**) y devuelve la respuesta. Detalles: - Modelo por defecto `gemini-2.5-pro` (con tu key de pago); alternativa económica `gemini-2.5-flash` . - **Cambio automático**: si pro se queda **sin saldo/cuota**, reintenta con flash; si falla todo (sin internet), usa el **cerebro local**. Jago nunca se queda mudo. - **"Pensamiento" ajustado** por modelo para respuestas directas y económicas. - **Modo kiosco**: la API Key NO se sube; se pega en el **Panel del profesor** (se guarda solo en ese navegador). Así la página pública nunca expone la clave.

● Fase 8 — Publicar en internet

El proyecto se subió a **GitHub** (`HAYS05/feria-jaguar`) y se activó **GitHub Pages** → enlace público gratis: <https://hays05.github.io/feria-jaguar/>. (Pasos en `GUIA-PUBLICAR.md` .)

+ Extra A — Agente conversacional (saludo guiado)

Tras escribir **nombre y apellido**, Jago mismo lleva la conversación (en el chat, con voz): 1. "¿Tienes un hijo o hija aquí en Saint Margaret?" → botones **Sí / No**. 2. Si sí → "¿En qué grado está?" → botones **Sexto...Undécimo**. 3. Jago **recomienda los proyectos de ese grado** (de `proyectos_por_grado` en `conocimiento.json`) y sigue la charla libre.

+ Extra B — Base de datos en la NUBE (Supabase)

`js/nube.js` guarda a **cada visitante y cada pregunta** en **Supabase**, en un solo lugar central, desde cualquier dispositivo. Usa la **API REST** con la **publishable key** (pública por diseño) y reglas **RLS** que solo permiten **GUARDAR** (nadie puede leer los datos desde la página; solo tú, desde el panel de Supabase). El **localStorage** queda como respaldo.

+ Extra C — Cola animada

La cola estaba **pintada en la foto**. Con **Python + Pillow** (`_tail_tool.py`) se **recortó** la cola y se creó una capa transparente (`assets/jaguar-cola.png`). En CSS, esa capa se **balancea** suavemente con `transform: rotate(...)` y un punto de giro en su base. La foto original queda debajo, así no se ve ninguna grieta.

+ Extra D — Rediseño profesional

Layout de **3 columnas** (acciones · avatar al centro · chat), **logo** en círculo blanco, **menú de proyectos por grado**, **código QR a la vista**, **fondo tecnológico** (`fondo-tech.svg`) y sombras. Todo pensado para **caber en una pantalla** sin scroll.

+ Extra E — Renombrado

El evento pasó de "Feria Tecnológica" a "Open House Saint Margaret School" en la página, el cartel y el cerebro de Jago.

7) Cómo se conectaron los servicios

🔑 A) Gemini (la IA)

1. Entrar a <https://aistudio.google.com/apikey> con tu cuenta Google.
2. **Create API key** (en un proyecto; el de nivel gratuito sirve, o uno con facturación para más límites).
3. **Modo kiosco**: abrir la página → **Panel del profesor** → pegar la key en "**Activar Gemini**". Se guarda solo en ese navegador.
4. El modelo se elige en `config.js` (`GEMINI_MODELO`).

⚠ La cuenta de Google tenía una **política de organización** que impide restringir la key por dominio, por eso se eligió el **modo kiosco** (la key nunca se publica).

🌐 B) GitHub Pages (publicar)

1. Crear el repositorio en GitHub (`feria-jaguar` , público).
2. Subir el proyecto con Git (`git push`).
3. **Settings** → **Pages** → **Deploy from a branch** → **main / root**.
4. Queda en <https://hays05.github.io/feria-jaguar/> (se actualiza solo con cada `git push`).

🗄 C) Supabase (base de datos en la nube)

1. Crear cuenta en <https://supabase.com> (puede ser con GitHub).
2. **New project** (`feria-jaguar`) + contraseña de base de datos.
3. **SQL Editor** → pegar y ejecutar el contenido de `datos/supabase-feria.sql` (crea las tablas `visitantes` y `mensajes` + seguridad RLS).
4. **Settings** → **API** → copiar **Project URL** y **publishable key**.
5. Pegarlas en `config.js` (`SUPABASE_URL` , `SUPABASE_ANON_KEY`).
6. Ver/exportar los datos en **Table Editor**.

8) Seguridad: cómo se protegió

- 🔑 **API Key de Gemini**: nunca se sube. Se usa en **modo kiosco** (solo en el navegador de la feria), con respaldo local.
- 🗄 **Supabase**: la **publishable key** es pública por diseño; la seguridad la dan las reglas **RLS** que **solo permiten INSERTAR** (nadie puede leer ni borrar datos desde la página).
- 🛡 **Datos de visitantes**: solo se pide **nombre, apellido, si tiene hijo en SMS y el grado**. Nunca datos sensibles (teléfono, dirección, contraseñas).
- 📦 **Respaldos**: el proyecto está versionado en **Git/GitHub** (historial completo) y hay **copias .zip** en la carpeta `backups/` (que no se sube).

9) Cómo reproducirlo desde cero (resumen)

1. Instalar **VS Code**, **Python**, **Git**, y usar **Chrome/Edge**.
 2. Crear las carpetas y archivos (HTML, CSS, JS) como en el [Mapa de archivos](#).
 3. Programar las fases 1→8 (voz, chat, micrófono, boca, datos, stands, IA).
 4. Llenar `datos/conocimiento.json` con la info real del evento.
 5. (Opcional IA) Crear API Key en **Google AI Studio** y activarla en modo kiosco.
 6. (Opcional nube) Crear proyecto en **Supabase**, correr `supabase-feria.sql`, pegar URL + key en `config.js`.
 7. Crear repo en **GitHub**, `git push`, activar **GitHub Pages**.
 8. Generar el **QR** apuntando a la URL pública e imprimir el `cartel.html`.
-

10) Glosario para principiantes

- **HTML / CSS / JavaScript**: esqueleto / diseño / cerebro de una página web.
 - **API**: forma en que dos programas se hablan (ej. pedirle al navegador "habla esto").
 - **API Key**: una "contraseña" que identifica tu cuenta al usar una API (Gemini). No se comparte.
 - **TTS / STT**: Texto a Voz / Voz a Texto.
 - **SVG**: dibujos hechos con código que se animan y nunca se pixelan.
 - **localStorage**: cajita del navegador que guarda datos en ESE dispositivo.
 - **Base de datos en la nube**: datos guardados en internet, accesibles desde cualquier dispositivo (Supabase).
 - **RLS (Row Level Security)**: reglas que dicen quién puede leer/escribir en cada tabla.
 - **Repositorio / Git / GitHub**: carpeta del proyecto con historial de cambios, guardada en la nube.
 - **Hosting / GitHub Pages**: servicio que pone tu página en internet.
 - **Modo kiosco**: una sola computadora (la del evento) donde se activa la IA con la key, sin exponerla.
-

11) Errores comunes y soluciones

-  **Cambié algo y no se ve**: recarga con **Ctrl + F5**.
 -  **No habla**: primero haz clic en un botón (los navegadores piden un clic antes de reproducir audio). Usa **Chrome/Edge**.
 -  **El micrófono no funciona**: da **permiso** y usa `localhost` o `https`.
 -  **"No pude cargar mi información"**: estás abriendo el archivo sin servidor → usa `INICIAR.bat`.
 -  **Gemini no responde**: revisa que activaste la key (Panel del profesor) y que hay internet; si no, Jago usa su cerebro local.
 -  **No se guardan visitantes en la nube**: revisa `SUPABASE_URL` y la key en `config.js`, y que corriste el SQL (tablas creadas).
-

✓ Resumen en una frase

Jago es una **página web gratis** (HTML + CSS + JS) donde un **jaguar habla, escucha, mueve la boca y la cola, conversa preguntando el grado del hijo**, responde con **IA (Gemini)**, **guarda a cada visitante en la nube (Supabase)** y está **publicado con GitHub Pages**, compartido por **código QR**.

Documento vivo: se actualiza al avanzar el proyecto. Última actualización: junio 2026.